

加減算

3

5

7

2

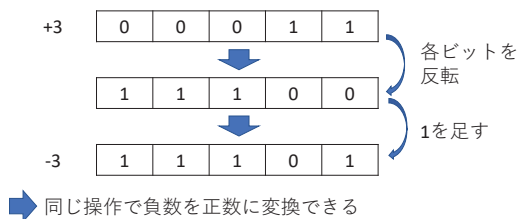
4

6

8

再掲 2の補数表現(2/3)

- 正の数は先頭に0を付加
- 負の数は先頭に1を付加し、各ビットを反転させてから1を加える
 - 例) $+3 \Rightarrow -3$ に変換



再掲 2の補数表現(3/3)

- 符号あり/なし10進数との対応

2進数	符号付き10進数	符号なし10進数
000	0	0
001	1	1
010	2	2
011	3	3
100	-4	4
101	-3	5
110	-2	6
111	-1	7

加算(1/4)

- 2進数の加算
- 4bitの符号なし整数
 - $10_{10} + 3_{10} = 13_{10}$
 - $1010_2 + 0011_2 = ?$

$$\begin{array}{r}
 \\
 + \\
 \hline

 \end{array}$$

$$1101_2 = 8_{10} + 4_{10} + 1_{10} = 13_{10}$$

手計算する時(筆算)と同じ手順で足せばよい

加算(2/4)

- 2進数の加算
- 4bitの符号あり整数
 - $-6_{10} + 3_{10} = -3_{10}$
 - $1010_2 + 0011_2 = ?$

$$\begin{array}{r}
 \\
 + \\
 \hline

 \end{array}$$

$$1101_2 = -8_{10} + 4_{10} + 1_{10} = -3_{10}$$

符号なしの場合と同じ手順で計算すればよい

加算(3/4)

- 2進数の加算
- 4bitの符号なし整数
 - $10_{10} + 6_{10} = 16_{10}$
 - $1010_2 + 0110_2 = ?$

$$\begin{array}{r}
 \\
 + \\
 \hline
 \textcircled{1}
 \end{array}$$

上位桁へのキャリーが発生

$$1010_2 + 0110_2 = 0000_2 ???$$

オーバーフローが発生するして結果が0になってしまった!

加算(4/4)

- 2進数の加算
- 4bitの符号あり整数
 - $3_{10} + 5_{10} = 8_{10}$
 - $0011_2 + 0101_2 = ?$

$$\begin{array}{r}
 \\
 + \\
 \hline

 \end{array}$$

上位桁へのキャリーがなくてもオーバーフロー

$$1000_2 = -8_{10}$$

オーバーフローが発生して正+正=負になってしまった!

加算時のキャリーとオーバーフロー

- 符号なし整数
 - キャリービット(MSBの一つ上の桁)が立つとオーバーフロー
- 符号あり整数
 - 加算する二つの数が同符号で、かつ加算結果が異符号となった場合にオーバーフロー
 - 加算する二つの数が異符号であればオーバーフローは発生しない
 - 4bitの場合(-8~7)
 - $-8 + (0 \sim 7) = -8 \sim -1$
 - $7 + (-8 \sim -1) = -1 \sim -6$

符号付き加算と符号なし加算

- 演算結果は同じ
 - $\Rightarrow$ MIPSのADD命令とADDU命令の違いは?
- オーバーフローを発生するかないかの差
 - C言語の場合、オーバーフローが発生しても無視するので、どちらも(オーバーフロー例外の発生しない)符号なしとして計算する


```

int a, b;
a=a+b;
addu $t0, $t0, $t1
unsigned int c, d;
c=c+d;
addu $t2, $t2, $t3
                    
```

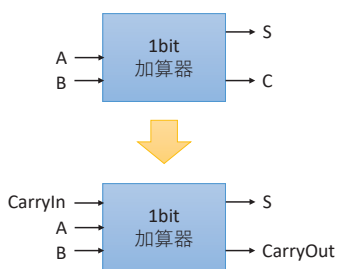
加算器

• 1bitの加算

	0	1
0	00	01
1	01	10

• 結果は2bitになる

- Sum: 下位1bit
- Carry: 上位1bit

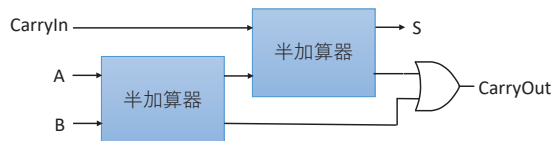


1bit加算器の中身

• ハーフアダー(Half Adder)



• 全加算器(Full Adder)

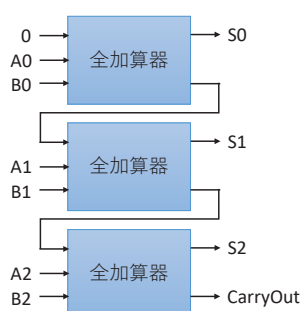


多ビット加算器

• 1bitの加算

	0	1
0	00	01
1	01	10

- 下位ビットのCarry Outを上位ビットのCarry Inに繋ぐ
- 最下位ビットのCarry Inは0



減算(1/2)

• 2進数の減算

• 4bitの符号なし整数

- $10_{10} - (+3_{10}) = 7_{10}$
- $1010_2 - (+0011_2) = ?$

	1	0	1	0
-	0	0	1	1
	0	1	1	1

$$0111_2 = 4_{10} + 2_{10} + 1_{10} = 7_{10}$$

手計算する時(筆算)と同じ手順で引けばよい

減算(2/2)

• 2進数の減算

• 4bitの符号あり整数

- $-2_{10} - (+3_{10}) = -5_{10}$
- $1110_2 - (+0011_2) = ?$

	1	1	1	0
-	0	0	1	1
	1	0	1	1

$$1011_2 = -8_{10} + 2_{10} + 1_{10} = -5_{10}$$

符号なしの場合と同じ手順で計算すればよい

減算器のアイディア

• $A - B = A + (-B)$

- 負数にするには、すべてのビットを反転させて1足せばよい

$$-A = \bar{A} + 1$$

• 減算器

- 符号逆転 + 加算(+1)

減算器

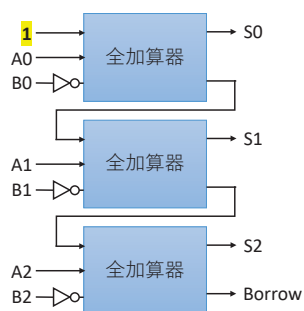
• Bのビットを反転

• $\bar{B}$ に+1する

⇒ CarryInを1にする

加減算器を作りたい

減算器は加算器に似てるので加算器に少し手を加える



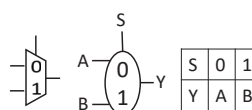
加減算器

加算:

CarryInを0にして、Bはそのまま

減算:

CarryInを1にして、Bのビットを反転する



マルチプレクサ (Multiplexer: MUX)、セクタ(Selector)